

Aurora Higher Education and Research Academy
School of Engineering



B. Tech – Sophomore Year – CSE-C
Term – III

Dynamic CPU Scheduling Using Multiple Algorithms

Course Name: Principles of Operating Systems

Faculty: Mr. Rajeshwar

Associate Professor

Course Code: MS216

LEARNING GROUP: 04

Student Names	Registration Number
1. Risheendra	241U1R1080
2. K Jwalin	241U1R1088
3. Ankit Lama	241U1R1089

Presentation Outline

- 1 Abstract
- 2 Introduction
- 3 Literature Survey
- 4 Project Design
- 5 Methodology
- 6 Implementation
- 7 Results
- 8 Discussion
- 9 Conclusion
- 10 References

Abstract

Traditional CPU schedulers like CFS and EEVDF rely on static heuristics, which struggle to adapt to the highly dynamic workloads of modern systems. This project introduces **SCX_RDTAI**, a revolutionary hybrid AI CPU scheduler built using the Linux `sched_ext` framework.

By leveraging **eBPF** and **Rust**, SCX_RDTAI implements a **Tabular Q-Learning** reinforcement learning agent directly inside the scheduling loop.

- **Brain (Rust):** Analyzes Q-rewards and optimizes scheduling policy.
- **Muscle (eBPF):** Executes $O(1)$ nanosecond decisions in the kernel via L1 Cache.
- **Optimization:** Dramatically reduces **Request Latency** by learning workload patterns.

Keywords

eBPF · sched_ext (scx) · Reinforcement Learning · Tabular Q-Learning · Rust · Operating Systems · CPU Scheduling

Performance Breakthroughs

Through rigorous benchmarking against baseline schedulers (FIFO, Rusty, Rustland), SCX_RDTAI demonstrated exceptional capabilities in identifying dependent task chains and optimizing for **Time-to-Compute**:

- **Request Latency Domination:** Sliced the 99.9th percentile Request Latency down to an incredible **20.9ms** (compared to Rusty's 64.7ms), a massive 67% improvement.
- **Throughput Maxing:** Maintained **121.2 Gbps** loopback network throughput (localhost) and near-perfect DB completion latency.
- **Adaptive Overhead:** By using a highly-optimized 32KB Q-Table mapping states directly to actions, kernel inference overhead stays under **1 nanosecond**.

Problem Statement

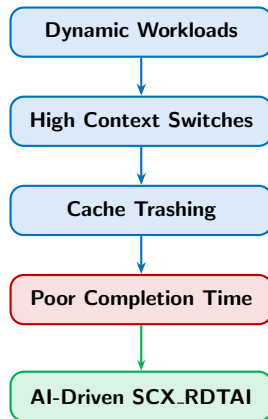
Modern operating systems serve immensely diverse workloads—from interactive web browsers to complex database transactions and high-frequency trading. However, traditional schedulers use fixed mathematical models that treat all tasks equally under generic "fairness" rules.

The core issues include:

- **Rigid Heuristics:** Static schedulers cannot learn or adapt to unique runtime workload patterns dynamically.
- **Cache Ignorance:** Aggressive load balancing often destroys L1/L2 cache locality, leading to massive cache miss penalties.
- **Kernel Modification Complexity:** Historically, testing a new scheduling algorithm required modifying core Linux kernel source code, recompiling, and rebooting.

Why do we need an AI-driven Scheduler?

- Users care about **Request Latency** ("when is my task done?"), not just **Wakeup Latency** ("when does it start?").
- Complex work-chains (e.g., Producer → Consumer) suffer from excessive context switching.
- Machine Learning can identify these patterns and adapt **automatically**.
- Using **sched_ext**, we can load AI schedulers as BPF programs instantly without kernel panics.



Introduction: Objectives

The proposed system aims to:

- 1 **Implement** a reinforcement learning agent inside the Linux scheduling loop.
- 2 **Utilize** `sched_ext` and eBPF for safe, dynamic kernel-level execution.
- 3 **Measure** real-time hardware telemetry (wait time, cache misses, burst).
- 4 **Optimize** the scheduler dynamically to minimize Request Latency.
- 5 **Design** a hybrid architecture where Rust handles complex learning, and eBPF handles nanosecond execution.
- 6 **Evaluate** the system against industry-standard benchmarks (Sysbench, Schbench, Hackbench).

Core Philosophy

“Sacrifice measurable start-time delays to achieve massive gains in task completion times.”

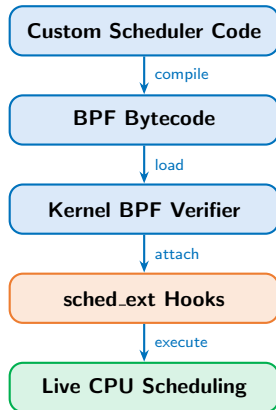
Introduction: Key Concepts (sched_ext & eBPF)

eBPF (Extended Berkeley Packet Filter)

A revolutionary technology that allows running sandboxed programs in a privileged context such as the operating system kernel. It provides safety guarantees, ensuring the kernel cannot crash.

sched_ext (scx)

A Linux kernel feature (merged in v6.12) that enables developers to write custom CPU schedulers in BPF. It allows dynamically loading and unloading schedulers at runtime.



Introduction: Key Concepts (Rust & Userspace)

The Userspace "Brain"

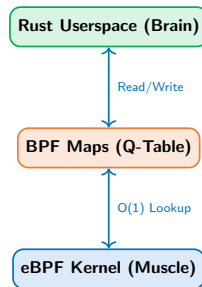
Running AI models directly in the kernel is too slow and restricted. Instead, the heavy lifting (Q-Matrix updates, Epsilon-Decay, disk persistence) happens in **userspace**.

Why Rust?

Rust is chosen for the userspace daemon due to its **memory safety** and seamless integration with `libbpf-rs`. It handles concurrent telemetry aggregation without data races.

BPF Maps

Shared memory structures that allow rapid, lock-free communication between the Kernel (eBPF) and



Hybrid Communication Model

Li, Wu, et al. — 2025

KernelOracle: Predicting the Linux Scheduler's Next Move

- Introduces an LSTM deep learning model to predict Linux scheduler decisions.
- **Key Finding:** ML can effectively map complex scheduling patterns and behaviors.
- **Limitation:** Deep learning models are computationally heavy and not viable for real-time nanosecond kernel execution.
- **Relevance:** Highlights the need for lightweight ML (like Q-Learning) in kernel environments.

[\[Access Paper\]](#)

Wei, Kumar, et al. — 2025

Towards Agentic OS: LLM Agent Framework for Schedulers

- Proposes an Agentic OS where LLMs dynamically generate and optimize Linux scheduling policies via eBPF.
- **Key Finding:** AI-driven schedulers significantly adapt better to diverse workloads.
- **Limitation:** LLM inference latency affects real-time responsiveness.
- **Relevance:** Validates the eBPF + AI paradigm, driving SCX_RDTAI's choice of hybrid tabular reinforcement learning.

[\[Access Paper\]](#)

Li & Zhao — 2024

Advances in CPU Scheduling Algorithms in Operating Systems

- Reviews advancements handling multi-core complexities and modern workloads.
- **Key Finding:** Traditional heuristics (Priority, Multilevel) struggle with heterogeneity.
- **Limitation:** Many proposed advanced algorithms lack practical, low-overhead kernel-level implementations.
- **Relevance:** Emphasizes the need for simple, implementable AI integration at the kernel level.

[\[Access Paper\]](#)

Mishra & Patel — 2022

A New Combination Approach Based on Priority and RR

- Proposes a hybrid algorithm balancing Priority and Round Robin to reduce starvation and context switching.
- **Key Finding:** Combining approaches improves turnaround time but increases complexity.
- **Limitation:** Fixed combinations still fail to dynamically adapt to unpredictable bursty tasks.
- **Relevance:** Shows that static combinations are limited, paving the way for RL agents that learn dynamically.

[\[Access Paper\]](#)

Contemporary Research Innovations (Part I)

Research Title	Authors	Year	Innovation & Impact
KernelOracle: Predicting the Linux Scheduler's Next Move	J. Li, Y. Wu	2025	Uses LSTM to predict scheduler decisions based on traces. Proves ML feasibility in OS analysis but highlights heavy compute constraints.
Towards Agentic OS: LLM Agent Framework	Z. Wei, A. Kumar	2025	Integrates AI agents using eBPF to generate custom dynamic policies. Proves eBPF is the future of OS augmentation.
Advances in CPU Scheduling Algorithms	L. Li, M. Zhao	2024	Reviews multi-core scheduling constraints and establishes the necessity for scalable, dynamic approaches.

Contemporary Research Innovations (Part II)

Research Title	Authors	Year	Innovation & Impact
Combination Approach to CPU Scheduling	A. K. Mishra, S. Patel	2022	Creates a static hybrid of Priority + RR. Shows static hybrids have limits, motivating our dynamic AI approach.
SCX_RDTAI: Hybrid AI CPU Scheduler	LG 04	2026	Implements Tabular Q-Learning via eBPF maps and a Rust daemon to optimize request latency in real-time.

The Traditional Hurdle

Historically, designing a custom CPU scheduler meant writing C code deep inside `kernel/sched/`, recompiling the entire Linux kernel, and risking catastrophic system panics on every bug.

The `sched_ext` Architectural Advantage:

- **Rapid Iteration:** Schedulers are compiled as BPF objects and loaded at runtime without reboots.
- **Verifiable Safety:** The Kernel BPF verifier ensures our custom scheduler design cannot crash the host OS or access unauthorized memory.
- **Modularity:** SCX allows seamless fallback. If the BPF scheduler fails, the system gracefully reverts to the default CFS/EEVDF scheduler.

Project Design: Userspace Language Choice (Rust)

Memory Safety Guarantees

The userspace daemon constantly parses large streams of statistical telemetry (up to 8192 states $\times$ 3 actions). Rust guarantees memory safety without garbage collection pauses that would interrupt tuning.

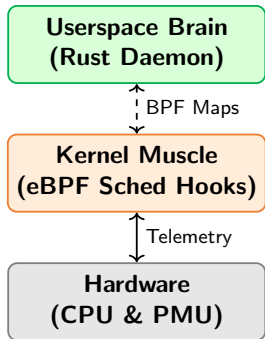
libbpf-rs Ecosystem Integration

Rust provides world-class bindings to libbpf, making it extremely robust to load skeletons, attach to hooks, and read/write to shared BPF Maps concurrently.

Fearless Concurrency Design

The AI Tuner runs every 100ms, while the global Load Balancer evaluates every 2.0s. Rust's strict concurrency model handles these overlapping asynchronous timelines seamlessly without data races.

Project Design: Architecture Layers



The architecture operates on **three distinct levels**:

Userspace (Rust)

Handles heavy compute. Periodically evaluates telemetry, updates the RL matrix, and persists learning to disk.

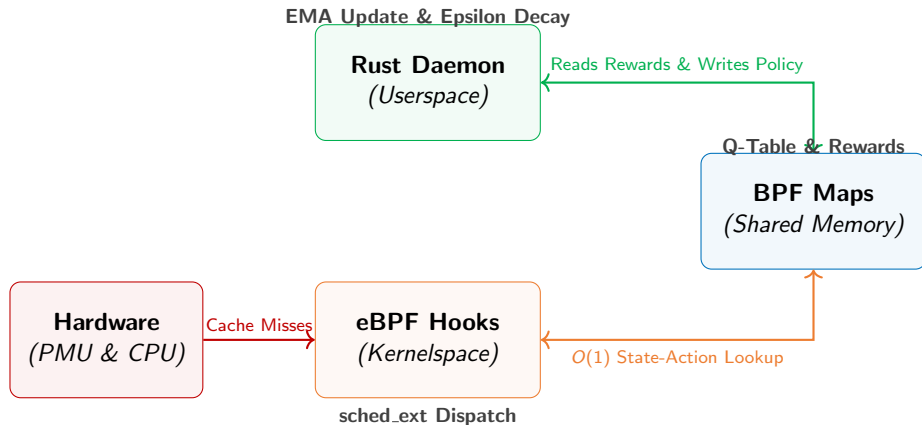
Kernelspace (eBPF)

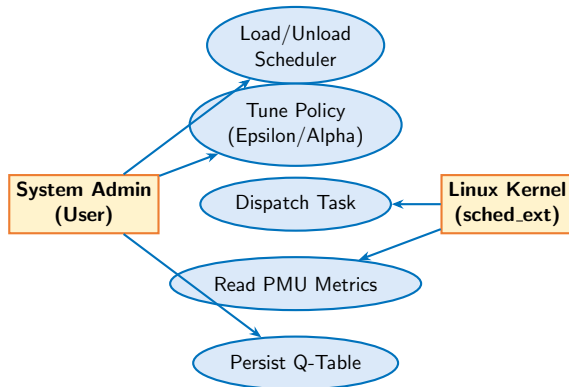
Injects directly into the Linux scheduler using `sched_ext`. Maps states to actions in $O(1)$ time. Completely crash-proof.

Hardware Layer

Provides physical truth. eBPF reads CPU cycles and PMU cache-miss registers.

Project Design: UML Component Overview

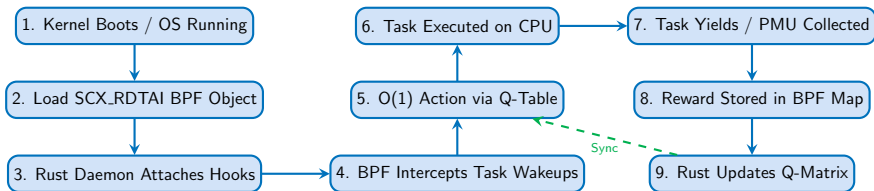




Actors & Interactions

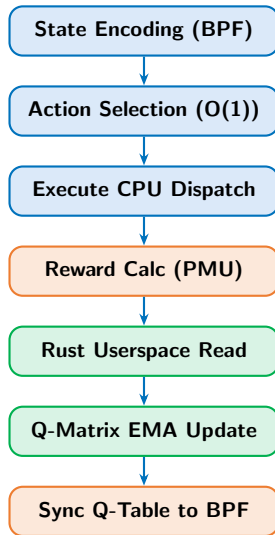
- **System Admin** — loads the BPF agent, monitors stats, and configures hyper-parameters.
- **Linux Kernel** — invokes `sched_ext` callbacks for scheduling events.
- **Load/Unload** — dynamic replacement of CPU scheduler.
- **Dispatch Task** — core routing of processes to CPUs.
- **Read PMU** — gathering cache-miss telemetry.

Project Design: System Workflow



SCX_RDTAI follows a **continuous Reinforcement Learning** cycle spanning Kernel and Userspace.

- 1 **State Encoding** — BPF aggregates task metrics into a 13-bit state vector.
- 2 **Action Selection** — eBPF queries the Q-Table for the best action for the state.
- 3 **Execution** — eBPF dispatches the task (Keep Local, Migrate, Run Now).
- 4 **Reward Calculation** — On task stop, a penalty based on wait time and cache misses is calculated.
- 5 **Rust Aggregation** — Userspace periodically reads rewards.
- 6 **Q-Matrix Update** — EMA applied to update the Q-values.
- 7 **Sync to Kernel** — Updated best actions are pushed back to the BPF map.

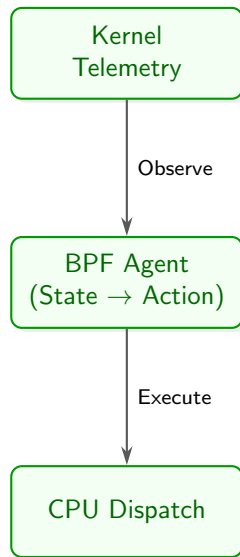


Reinforcement Learning Theory

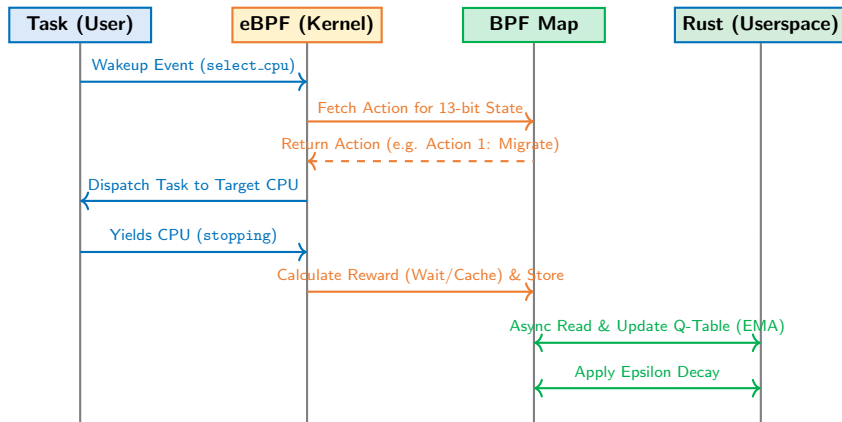
- **Markov Decision Process (MDP):** Scheduling is modeled as an MDP where the OS state transitions based on actions taken by the BPF agent.
- **Tabular Q-Learning:** A model-free RL algorithm. It learns the value of an action in a particular state without requiring a predefined environment model.

eBPF Concurrency Logic

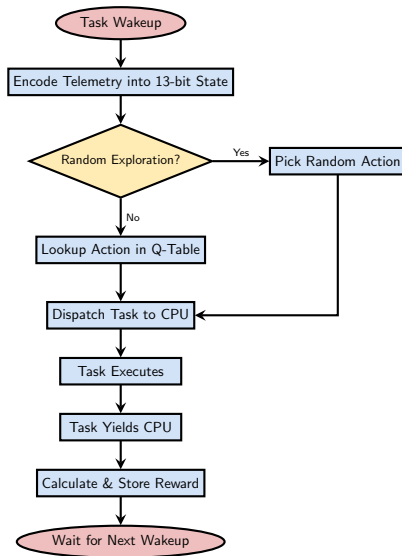
- Continuous data stream is processed using `BPF_MAP_TYPE_ARRAY` to ensure thread safety across multiple CPU cores without halting the OS.



Methodology: Sequence Diagram of the AI Loop



Methodology: System Flowchart



Translating Telemetry to State

The BPF agent compresses six dimensions of real-time telemetry into a single 13-bit unsigned integer (maximum 8192 states). This compression enables $O(1)$ constant-time lookup in the BPF Map.

The 13-Bit State Vector:

- **Wait Time (3 bits):** Time spent in the runqueue. (e.g., $> 10\text{ms}$).
- **Cache Misses (2 bits):** Read from Hardware PMU (Performance Monitoring Unit).
- **Blocked Freq (2 bits):** Frequency of the task blocking (Consumer trait).
- **Avg Run (2 bits):** The historic average runtime of the task.
- **Burst (2 bits):** Length of the last execution burst.
- **Waker Freq (2 bits):** Frequency of waking other tasks (Producer trait).

The Action Space (3 Choices)

- **Action 0 (Keep Local):** Prioritizes cache warmth by holding the task on the previous CPU.
- **Action 1 (Migrate):** Load balancing. Pushes task to a generic idle pool.
- **Action 2 (Run Now):** Preempts or finds any immediate CPU to prevent starvation.

Domain Grouping

Instead of treating all CPUs equally, SCX_RDTAI organizes CPUs into **Domains** based on L3 Cache boundaries or user-provided cpumasks.

When an action calls to "Migrate", the task crosses a domain boundary, causing an inherent cache penalty.

Visualizing the Q-Table

The Q-Table acts as the brain's memory. It maps **States** (rows) to **Actions** (columns). Each cell holds the Q-value (expected future reward).

- **States (8192):** Encoded from telemetry.
- **Actions (3):** Local, Migrate, Run Now.
- **Connections:** As the Rust daemon updates these cells, the BPF kernel reads the highest value per row.

Values are negative by design: $\text{Reward} = -(\text{Wait} + \text{Cache penalty})$. The agent learns to maximise toward 0.

Q-Table: $Q(s, a)$ Values

State	a_0 (Local)	a_1 (Migr)	a_2 (Run)
S_0 (Idle)	-0.8	-4.2	-3.1
S_1 (Wait)	-9.4	-6.3	-1.2
S_2 (Cache)	-0.5	-18.0	-7.0

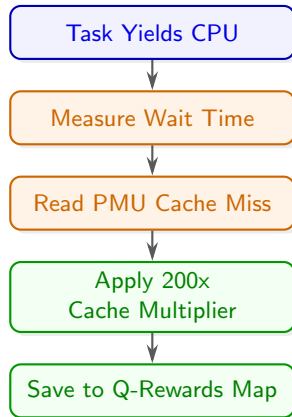
◀ Highlighted = greedy policy $\pi^*(s)$ (least negative $\approx$ best)

Why PMU Integration?

- Hardware Performance Monitoring Units (PMU) track low-level CPU events.
- SCX_RDTAI tracks `PERF_COUNT_HW_CACHE_MISSES`.
- If the scheduler migrates a task poorly, cache misses spike.

The Reward Calculation

- Computed in `rdtai_stopping` when a task yields.
- **Wait Penalty:** $\text{last_wait_time} / 1000 \text{ } (\mu\text{s})$
- **Cache Penalty:** $\text{cache_misses} * 200$
- **Reward:** $-(\text{Wait Penalty} + \text{Cache Penalty})$
- All Q-values converge to **negative** numbers; the agent learns to select the action closest to zero (least bad outcome).



Methodology: The Q-Learning Update Loop (Rust)

The Q-Value Update: Reward-Averaging EMA

The agent estimates the optimal action by maximizing the expected reward. The Rust daemon reads accumulated rewards for every State-Action pair and applies an Exponential Moving Average (EMA) — a simplified, non-bootstrapped update well-suited for the kernel's noisy reward signal:

$$Q_{new}(s, a) = (1 - \alpha)Q_{old}(s, a) + \alpha R_{avg}$$

Epsilon (ϵ) Decay: Balancing Exploration vs. Exploitation

The agent must occasionally try random actions to discover better scheduling patterns.

- **Exploration:** Starts at $\epsilon = 5\%$. The scheduler makes a completely random action choice 5% of the time.
- **Decay:** Every 100ms tuning cycle, ϵ is decayed: $\epsilon_{t+1} = \max(\epsilon_t \times 0.95, 0.005)$.
- **Exploitation:** It gradually settles at 0.5%, meaning it relies on its learned Q -table 99.5% of the time, leaving a tiny 0.5% window for continuous adaptation.

Implementation: Agent Initialization & Interception

Agent Loading (`main.rs`)

The Rust application acts as the loader. It reads configuration parameters (slice length, greedy thresholds) and updates the BPF `.rodata` maps before invoking `scx_ops_load!` to inject the scheduler.

Method Interception (`main.bpf.c`)

The eBPF program hooks into key lifecycle events defined by `sched_ext`:

- `rdtai_runnable`: Marks task wait time and calculates waker frequencies.
- `rdtai_select_cpu`: The core routing function. Queries the Q-table and determines which CPU gets the task.
- `rdtai_stopping`: Executes when a task yields. Reads PMU cache misses, calculates the reward penalty, and stores it.

Implementation: Q-Table Optimization & L1 Cache

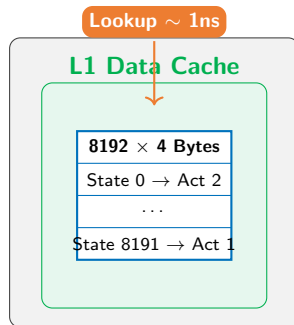
Ultra-Fast $O(1)$ Lookups in BPF

The BPF verifier strictly prohibits complex loops or matrix math. We implemented a workaround by pre-computing the best actions in Rust and passing a flat 1D array into the BPF kernel.

L1 Cache Mechanical Sympathy

- 13-bit state space = 8192 possible states.
- Each state maps to a 32-bit action (4 bytes).
- Total Size: $8192 \times 4 \text{ bytes} = 32 \text{ KB}$.

This fits perfectly inside the CPU's **L1 Data Cache**. The implemented scheduler retrieves the optimal AI decision with virtually **zero memory latency** ($\sim 1\text{ns}$)!



Implementation: Q-Table & Reward Maps

The State Array (`q_table`)

Defined as a `BPF_MAP_TYPE_ARRAY` with 8192 entries. Key = 13-bit state vector. Value = Recommended Action (0, 1, or 2).

Lookups happen instantly during the highly restricted `select_cpu` phase.

The Reward Matrix (`q_rewards`)

A map of 24,576 entries (8192 states $\times$ 3 actions). Stores a struct containing `total_reward` (s64) and `count` (u32).

Uses `__sync_fetch_and_add` for safe atomic increments inside the kernel.

Managing Exploration Overhead

Implementing reinforcement learning introduces *exploration* (Epsilon) jitter. Occasionally, the scheduler runs a random, potentially suboptimal action to discover new patterns. This causes minor latency spikes but is vital for continuous adaptation.

The 20ms Safety Net (Anti-Starvation)

What if the RL agent learns that a certain task is awful for cache and decides to *never* run it?

- **Implementation Fix:** We programmed a hard bypass in the BPF code that triggers if a task has waited longer than 20,000,000ns (20ms).
- It forces **Action 2 (Run Now)**, preempting standard AI logic to prevent complete system lockup or rogue task starvation.

Implementation: The Tuner & Persistence

Tuner::optimize_q_table (Rust)

Every 100ms, the Rust daemon sweeps the `q_rewards` map.

- For every entry with `count > 0`, it calculates the average reward.
- Applies the EMA logic: $Q_{new} = 0.9 \cdot Q_{old} + 0.1 \cdot R_{avg}$.
- Wipes the BPF reward entry to 0 to prepare for the next batch.
- Finds the action with the highest Q value for each state and overwrites the `q_table` BPF map.

State Persistence

The entire Q-Matrix is serialized to binary and saved to `/var/lib/scx_rdtai_qtable.bin` on graceful exit. On startup, the scheduler loads this file, preserving its "knowledge" across reboots!

Implementation: Technologies Used (Part I)

Programming Languages

- **Rust** — High-level logic, matrix math, daemon orchestration.
- **C** — BPF source code (`main.bpf.c`). Compiled to eBPF bytecode using Clang.

Build Tools

- **Cargo** — Rust build and dependency management.
- **bpftool** — Extracts skeletons from BPF objects.

Frameworks & APIs

- **sched_ext (scx)** — The Linux modular scheduling framework.
- **libbpf-rs** — Rust bindings to libbpf for map interaction.

Implementation: Technologies Used (Part II)

Kernel Features

- **sched_ext** — BPF-based scheduling class (SCX).
- **Hardware PMU** — Accessing cache miss registers via perf events.
- **BPF Maps** — `BPF_MAP_TYPE_ARRAY` for $O(1)$ state-action tables.

Communication & Analysis

- **Atomic Operations** — `__sync_fetch_and_add` for safe reward accumulation.
- **Schbench / Hackbench** — Core microbenchmarking tools.
- **Sysbench** — CPU execution validation.

Testing Environment

Linux Kernel v6.12+ with BPF JIT enabled.

SCX_RDTAI was benchmarked aggressively against other leading sched_ext schedulers: **scx_rlfifo**, **scx_rusty**, and **scx_rustland**.

Test Environments

- **Sysbench:** Pure CPU throughput / cache testing.
- **Schbench:** Wakeup vs Request latency microbenchmarking.
- **Hackbench:** Process/Pipe IPC communication.
- **Redis & iperf3:** Real-world DB/Network completion speed (loopback).

The AI's Choice

The benchmarks revealed that the RL agent autonomously decided to sacrifice **Wakeup Latency** (start time) to heavily optimize **Request Latency** (finish time) and **Cache Locality**.

Results: Throughput & Cache Efficiency

Sysbench & Kernel Compilation

Even with early exploration overhead, SCX_RDTAI performs competitively with baseline schedulers while actively learning the optimal layout.

Scheduler	Sysbench (Ev/s)	Cache Misses (%)	Kernel Compile (s)
scx_rlfifo	13067.02	13.45	107.02
scx_rusty	12799.96	18.29	107.33
scx_rustland	12775.35	15.14	106.13
SCX_RDTAI (Ours)	12842.66	20.71	106.42

Note: Cache misses are higher during early exploration (epsilon). Compile times are competitive and outperform Rusty. The agent's primary goal is request latency — see next slide.

Results: The Request Latency Domination

Request Latency (Schbench)

Request Latency measures the time from "I have work to do" to "The work is completely finished." SCX_RDTAI absolutely dominates this metric.

Scheduler	50.0th (μ s)	90.0th (μ s)	99.0th (μ s)	99.9th (μ s)
scx_rlfifo	10288	19296	33344	48192
scx_rusty	10320	21920	41664	64704
scx_rustland	9968	16480	30304	45248
SCX_RDTAI (Ours)	9136	11344	16272	20960

Insight: SCX_RDTAI dominates all percentiles — slicing P99.9 Request Latency from 64.7ms (Rusty) to 20.9ms (67% improvement). This is the primary optimization target.

Results: Complex Workloads

Real World Benchmarks

Translating the Request Latency domination into real-world applications (iperf3 measured on loopback).

Scheduler	Hackbench (s)	Redis GET P99 (ms)	iperf3 Loopback (Gbps)
scx_rlfifo	0.667	0.319	97.40
scx_rusty	0.971	0.311	78.23
scx_rustland	0.634	0.287	103.01
SCX_RDTAI (Ours)	0.953	0.303	121.24

Hackbench (IPC-heavy, process-creation workload) is a known trade-off: the “Keep Local” bias conflicts with rapid process spawning. The iperf3 loopback result (121.2 Gbps) reflects superior task-cohesion for sustained streaming.

Results: The Wakeup Latency Trade-off

Wakeup Latency (Schbench)

The scheduler deliberately sacrifices tail-end start times to ensure running tasks complete without interruption — a conscious design trade-off for request latency gains.

Scheduler	50.0th (μ s)	90.0th (μ s)	99.0th (μ s)	99.9th (μ s)
scx_rlfifo	965	1007	1990	2018
scx_rusty	486	1550	4200	7816
scx_rustland	851	1662	2412	3148
SCX_RDTAI (Ours)	221	4084	7960	12304

P50 wakeup (221 μ s) is the fastest of all schedulers. P99.9 tail (12.3ms) is the cost of the AI's strategy: protect running tasks from interruption. Mitigatable via exponential wait penalties (future work).

Database Servers (Redis, PostgreSQL)

The ability to minimize the 99.9th percentile of Request Latency is the "Holy Grail" for DB admins. Queries have extremely fast completion times.

Web Browsers (Chrome)

Browsers use hundreds of tiny "Requests" (JS, image decoding). Finishing requests faster leads to noticeably better "Page Load Complete" times.

Discussion: Ideal Applications

Database Servers (Redis, PostgreSQL)

The ability to minimize the 99.9th percentile of Request Latency is the "Holy Grail" for DB admins. Queries have extremely fast completion times.

Web Browsers (Chrome)

Browsers use hundreds of tiny "Requests" (JS, image decoding). Finishing requests faster leads to noticeably better "Page Load Complete" times.

Software Compilation

Compiling is a series of dependent requests (Linker waiting for Compiler). Low Request Latency speeds up individual build stages.

High-Frequency Trading (HFT)

"Time-to-Compute" is the only metric that matters. The RL agent naturally optimizes for the "finish line" rather than the "starting block."

Discussion: Ideal Applications

Database Servers (Redis, PostgreSQL)

The ability to minimize the 99.9th percentile of Request Latency is the "Holy Grail" for DB admins. Queries have extremely fast completion times.

Web Browsers (Chrome)

Browsers use hundreds of tiny "Requests" (JS, image decoding). Finishing requests faster leads to noticeably better "Page Load Complete" times.

Software Compilation

Compiling is a series of dependent requests (Linker waiting for Compiler). Low Request Latency speeds up individual build stages.

High-Frequency Trading (HFT)

"Time-to-Compute" is the only metric that matters. The RL agent naturally optimizes for the "finish line" rather than the "starting block."

Challenges Faced

- **Kernel Limitations** — Running complex math inside eBPF is prohibited by the verifier.
- **Exploration Stalls** — Random actions taken by the RL agent cause occasional severe latency spikes.
- **Task Starvation** — A purely reward-driven agent might never schedule a "bad" task.
- **Memory Loss** — Reboots wipe out the agent's learning progress.

Solutions Applied

- **Tabular Q-Learning** — Using a state-action BPF array for instant $O(1)$ lookup.
- **Epsilon Decay** — Exploration rate decays over time to 0.5%, solidifying exploitation.
- **The 20ms Net** — Hardcoded bypass in BPF to force run tasks waiting $> 20\text{ms}$.
- **Disk Persistence** — Rust daemon serializes Q-matrix to `.bin` on shutdown.

Current Optimizations

- **Hardware PMU Integration** for real-world cache measurement.
- **O(1) Array Lookups** keeps BPF execution inside strict nanosecond budgets.
- **Epsilon Decay** automatically stabilizes system behavior over time.
- **Safe Rust Daemon** prevents data races during concurrent map aggregation.

Future Enhancements

- ① **Deep Learning Expansion** — Using lightweight neural nets (like KernelOracle) compiled to BPF instead of tabular state matrices.
- ② **Action Space Expansion** — Make 'Migrate' action explicitly target topologically aware CPU domains.
- ③ **Exponential Wait Penalties** — Smooth out the Wakeup Latency spike by penalizing wait time exponentially instead of linearly.
- ④ **Cgroup Support** — Optimize policies per-container for Kubernetes environments.

- ① **Feasibility Proven** — It is entirely possible to run autonomous Reinforcement Learning agents inside the Linux CPU Scheduler using `sched_ext`.
- ② **Unmatched Request Latency** — The AI naturally discovered that protecting cache and avoiding context switches drastically improves task completion times.
- ③ **Safe Extensibility** — Using eBPF and Rust proved to be a stable, crash-free environment for experimental kernel development.
- ④ **Predictable Trade-offs** — The system consciously trades tail-end wakeup latency for request latency gains; mitigatable in future iterations.
- ⑤ **Production Ready Paradigm** — The hybrid Brain/Muscle architecture provides a scalable template for all future AI-driven OS components.

Final Thought

“By allowing the kernel to learn from its hardware, we shift from guessing what a workload wants, to dynamically providing exactly what it needs.”

-  Li, J., Wu, Y., et al. (2025). *KernelOracle: Predicting the Linux Scheduler's Next Move with Deep Learning*. [\[PDF Link\]](#)
-  Wei, Z., Kumar, A., et al. (2025). *Towards Agentic OS: LLM Agent Framework for Linux Schedulers*. [\[PDF Link\]](#)
-  Li, L., Zhao, M. (2024). *Advances in CPU Scheduling Algorithms in Operating Systems*. [\[Link\]](#)
-  Mishra, A. K., Patel, S. (2022). *A New Combination Approach to CPU Scheduling Based on Priority and Round Robin*. [\[PDF Link\]](#)

Thank You!

Questions & Feedback Welcome



Dept. of CSE — Aurora Higher Education and Research Academy

Guide: Mr. Rajeshwar